CS 171 / EE 142: Intro to ML and DM

Christian Shelton

UC Riverside

Slide Set 13: SVMs and Kernels

Slides from CS 171 / EE 142

- From UC Riverside
  - CS 171 / EE 142: Introduction to Machine Learning and Data Mining
  - Professor Christian Shelton
- DO NOT REDISTRIBUTE
  - For use only by enrolled students in the course

Non-linear SVM

Recall the dual QP:

$$\begin{aligned} \underset{a_1,\ldots,a_m}{\arg\min} \quad & \frac{1}{2}\vec{a}^{\mathsf{T}}\boldsymbol{G}\vec{a} - \vec{a}^{\mathsf{T}}\vec{1} \\ \text{s.t.} \quad & 0 \leq a_i \leq C \qquad \text{for all } i \\ & \vec{a}^{\mathsf{T}}\boldsymbol{Y} = 0 \end{aligned}$$

depends on the data only through the matrix $\boldsymbol{G}$:

$$G_{ij} = y_i \vec{x}_i{}^{\mathsf{T}} \vec{x}_j y_j$$

Non-linear SVM

Recall the dual QP:

$$\begin{aligned} \underset{a_1,\ldots,a_m}{\arg\min} \quad & \frac{1}{2}\vec{a}^{\mathsf{T}}\boldsymbol{G}\vec{a} - \vec{a}^{\mathsf{T}}\vec{1} \\ \text{s.t.} \quad & 0 \leq a_i \leq C \qquad \text{for all } i \\ & \vec{a}^{\mathsf{T}}\boldsymbol{Y} = 0 \end{aligned}$$

depends on the data only through the matrix $\boldsymbol{G}$:

$$G_{ij} = y_i \vec{x}_i{}^{\mathsf{T}} \vec{x}_j y_j$$

If we use a feature map:

$$G_{ij} = y_i \phi(\vec{x}_i)^{\mathsf{T}} \phi(\vec{x}_j) y_j$$

Non-linear SVM

Recall the dual QP:

$$\begin{aligned} \underset{a_1,\ldots,a_m}{\arg\min} \quad & \frac{1}{2}\vec{a}^{\mathsf{T}}\boldsymbol{G}\vec{a} - \vec{a}^{\mathsf{T}}\vec{1} \\ \text{s.t.} \quad & 0 \leq a_i \leq C \qquad \text{for all } i \\ & \vec{a}^{\mathsf{T}}\boldsymbol{Y} = 0 \end{aligned}$$

depends on the data only through the matrix $\boldsymbol{G}$:

$$G_{ij} = y_i \vec{x}_i{}^{\mathsf{T}} \vec{x}_j y_j$$

If we use a feature map:

$$G_{ij} = y_i \phi(\vec{x}_i)^{\mathsf{T}} \phi(\vec{x}_j) y_j$$

No other change necessary.

- Numbers of variables and constraints do not change
- No matter how many features we add!

Kernel

We call the inner product of two feature vectors the **kernel**:

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^{\mathsf{T}} \phi(\vec{x}')$$

And then

$$G_{ij} = y_i k(\vec{x}_i, \vec{x}_j) y_j$$

Kernel

We call the inner product of two feature vectors the **kernel**:

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^{\mathsf{T}} \phi(\vec{x}')$$

And then

$$G_{ij} = y_i k(\vec{x}_i, \vec{x}_j) y_j$$

So, SVMs only depend on the kernel between pairs of points.

Kernel Benefit

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^{\mathsf{T}} \phi(\vec{x}')$$

Why is this helpful?

Kernel Benefit

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\mathsf{T} \phi(\vec{x}')$$

Why is this helpful?

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}$$

Kernel Benefit

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\mathsf{T} \phi(\vec{x}')$$

Why is this helpful?

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}$$

$$k(\vec{x}, \vec{x}') = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}^\mathsf{T} \begin{bmatrix} 1 \\ \sqrt{2}x_1' \\ \sqrt{2}x_2' \\ \sqrt{2}x_3' \\ \sqrt{2}x_1'x_2' \\ \sqrt{2}x_1'x_3' \\ \sqrt{2}x_2'x_3' \\ x_1'^2 \\ x_2'^2 \\ x_3'^2 \end{bmatrix}$$

Kernel Benefit

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\mathsf{T} \phi(\vec{x}')$$

Why is this helpful?

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}$$

$$k(\vec{x}, \vec{x}') = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}^\mathsf{T} \begin{bmatrix} 1 \\ \sqrt{2}x_1' \\ \sqrt{2}x_2' \\ \sqrt{2}x_3' \\ \sqrt{2}x_1'x_2' \\ \sqrt{2}x_1'x_3' \\ \sqrt{2}x_2'x_3' \\ {x_1'}^2 \\ {x_2'}^2 \\ {x_3'}^2 \end{bmatrix}$$

$$\begin{aligned} &= 1 + 2x_1x_1' + 2x_2x_2' + 2x_3x_3' \\ &\quad + 2x_1x_1'x_2x_2' + 2x_1x_1'x_3x_3' + 2x_2x_2'x_3x_3' \\ &\quad + x_1^2{x_1'}^2 + x_2^2{x_2'}^2 + x_3^2{x_3'}^2 \end{aligned}$$

Kernel Benefit

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\mathsf{T} \phi(\vec{x}')$$

Why is this helpful?

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}$$

$$k(\vec{x}, \vec{x}') = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}^\mathsf{T} \begin{bmatrix} 1 \\ \sqrt{2}x_1' \\ \sqrt{2}x_2' \\ \sqrt{2}x_3' \\ \sqrt{2}x_1'x_2' \\ \sqrt{2}x_1'x_3' \\ \sqrt{2}x_2'x_3' \\ {x_1'}^2 \\ {x_2'}^2 \\ {x_3'}^2 \end{bmatrix}$$

$$\begin{aligned} &= 1 + 2x_1x_1' + 2x_2x_2' + 2x_3x_3' \\ &\quad + 2x_1x_1'x_2x_2' + 2x_1x_1'x_3x_3' + 2x_2x_2'x_3x_3' \\ &\quad + x_1^2{x_1'}^2 + x_2^2{x_2'}^2 + x_3^2{x_3'}^2 \\ &= (1 + x_1x_1' + x_2x_2' + x_3x_3')^2 \end{aligned}$$

Kernel Benefit

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\mathsf{T} \phi(\vec{x}')$$

Why is this helpful?

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}$$

$$k(\vec{x}, \vec{x}') = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}^\mathsf{T} \begin{bmatrix} 1 \\ \sqrt{2}x_1' \\ \sqrt{2}x_2' \\ \sqrt{2}x_3' \\ \sqrt{2}x_1'x_2' \\ \sqrt{2}x_1'x_3' \\ \sqrt{2}x_2'x_3' \\ x_1'^2 \\ x_2'^2 \\ x_3'^2 \end{bmatrix}$$

$$\begin{aligned} &= 1 + 2x_1x_1' + 2x_2x_2' + 2x_3x_3' \\ &\quad + 2x_1x_1'x_2x_2' + 2x_1x_1'x_3x_3' + 2x_2x_2'x_3x_3' \\ &\quad + x_1^2x_1'^2 + x_2^2x_2'^2 + x_3^2x_3'^2 \\ &= (1 + x_1x_1' + x_2x_2' + x_3x_3')^2 \\ &= (1 + \vec{x}^\mathsf{T}\vec{x}')^2 \end{aligned}$$

Kernel Benefit

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\mathsf{T} \phi(\vec{x}')$$

Why is this helpful?

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}$$

$k(\vec{x}, \vec{x}')$ can be calculated without forming $\phi(\vec{x})$!

$$k(\vec{x}, \vec{x}') = \begin{bmatrix} 1 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_3 \\ \sqrt{2}x_1x_2 \\ \sqrt{2}x_1x_3 \\ \sqrt{2}x_2x_3 \\ x_1^2 \\ x_2^2 \\ x_3^2 \end{bmatrix}^\mathsf{T} \begin{bmatrix} 1 \\ \sqrt{2}x_1' \\ \sqrt{2}x_2' \\ \sqrt{2}x_3' \\ \sqrt{2}x_1'x_2' \\ \sqrt{2}x_1'x_3' \\ \sqrt{2}x_2'x_3' \\ {x_1'}^2 \\ {x_2'}^2 \\ {x_3'}^2 \end{bmatrix}$$

$$\begin{aligned} &= 1 + 2x_1x_1' + 2x_2x_2' + 2x_3x_3' \\ &\quad + 2x_1x_1'x_2x_2' + 2x_1x_1'x_3x_3' + 2x_2x_2'x_3x_3' \\ &\quad + x_1^2{x_1'}^2 + x_2^2{x_2'}^2 + x_3^2{x_3'}^2 \\ &= (1 + x_1x_1' + x_2x_2' + x_3x_3')^2 \\ &= (1 + \vec{x}^\mathsf{T}\vec{x}')^2 \end{aligned}$$

SVM Predictions with Kernels

$$\widetilde{f}(\vec{x}) = b + \vec{w}^{\mathsf{T}}\vec{x}$$

SVM Predictions with Kernels

$$\begin{aligned}\widetilde{f}(\vec{x}) &= b + \vec{w}^{\mathsf{T}}\vec{x} \\ &= b + \sum_{i=1}^{m} a_i y_i \vec{x}_i{}^{\mathsf{T}}\vec{x}\end{aligned}$$

SVM Predictions with Kernels

$$\begin{aligned}\widetilde{f}(\vec{x}) &= b + \vec{w}^{\mathsf{T}}\phi(\vec{x}) \\ &= b + \sum_{i=1}^{m} a_i y_i \phi(\vec{x}_i)^{\mathsf{T}}\phi(\vec{x})\end{aligned}$$

SVM Predictions with Kernels

$$\begin{aligned}\widetilde{f}(\vec{x}) &= b + \vec{w}^{\mathsf{T}}\phi(\vec{x}) \\ &= b + \sum_{i=1}^{m} a_i y_i \phi(\vec{x}_i)^{\mathsf{T}}\phi(\vec{x}) \\ &= b + \sum_{i=1}^{m} a_i y_i k(\vec{x}_i, \vec{x})\end{aligned}$$

SVM Predictions with Kernels

$$\begin{aligned}\widetilde{f}(\vec{x}) &= b + \vec{w}^{\mathsf{T}}\phi(\vec{x}) \\ &= b + \sum_{i=1}^{m} a_i y_i \phi(\vec{x}_i)^{\mathsf{T}}\phi(\vec{x}) \\ &= b + \sum_{i=1}^{m} a_i y_i k(\vec{x}_i, \vec{x})\end{aligned}$$

Learning and prediction only depend on k, not on ϕ.

Kernel SVM

Learning:

- Form $\boldsymbol{G}$: $G_{ij} = y_i y_j k(\vec{x}_i, \vec{x}_j)$
- Solve

$$\begin{aligned} \underset{a_1,\ldots,a_m}{\arg\min} \quad & \frac{1}{2}\vec{a}^{\mathsf{T}}\boldsymbol{G}\vec{a} - \vec{a}^{\mathsf{T}}\vec{1} \\ \text{s.t.} \quad & \vec{0} \le \vec{a} \le C\vec{1} \qquad \text{for all } i \\ & \vec{a}^{\mathsf{T}}\boldsymbol{Y} = 0 \end{aligned}$$

- Store $a_1, \ldots, a_m$ [most are 0]
- Calculate b from $a_1, \ldots, a_m$

Prediction:

- Calculate $\widetilde{f}(\vec{x}) = b + \sum_{i=1}^{m} a_i y_i k(\vec{x}_i, \vec{x})$
- Predict sign of $\widetilde{f}(\vec{x})$

Kernel SVM

Learning:

- Form $\boldsymbol{G}$: $G_{ij} = y_i y_j k(\vec{x}_i, \vec{x}_j)$
- Solve

$$\begin{aligned} \underset{a_1,\ldots,a_m}{\arg\min} \quad & \frac{1}{2}\vec{a}^\mathsf{T}\boldsymbol{G}\vec{a} - \vec{a}^\mathsf{T}\vec{1} \\ \text{s.t.} \quad & \vec{0} \le \vec{a} \le C\vec{1} \qquad \text{for all } i \\ & \vec{a}^\mathsf{T}\boldsymbol{Y} = 0 \end{aligned}$$

- Store $a_1, \ldots, a_m$ [most are 0]
- Calculate b from $a_1, \ldots, a_m$

Hyper-parameters: C, $k(\cdot, \cdot)$

Prediction:

- Calculate $\widetilde{f}(\vec{x}) = b + \sum_{i=1}^{m} a_i y_i k(\vec{x}_i, \vec{x})$
- Predict sign of $\widetilde{f}(\vec{x})$

Possible Kernels

Any function $k(\cdot, \cdot)$ that corresponds to

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^{\mathsf{T}} \phi(\vec{x}')$$

is valid to use.

Possible Kernels

Any function $k(\cdot, \cdot)$ that corresponds to

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^{\mathsf{T}} \phi(\vec{x}')$$

is valid to use.

Popular ones:

Linear $k(\vec{x}, \vec{x}') = \vec{x}^{\mathsf{T}} \vec{x}'$

Possible Kernels

Any function $k(\cdot, \cdot)$ that corresponds to

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^{\mathsf{T}} \phi(\vec{x}')$$

is valid to use.

Popular ones:

Linear $k(\vec{x}, \vec{x}') = \vec{x}^{\mathsf{T}} \vec{x}'$

Polynomial $k(\vec{x}, \vec{x}') = (c + \vec{x}^{\mathsf{T}} \vec{x}')^d$

[hyper-parameters: c, d]

Corresponds to all polynomial features up through degree d

Possible Kernels

Any function $k(\cdot, \cdot)$ that corresponds to

$$k(\vec{x}, \vec{x}') = \phi(\vec{x})^\mathsf{T} \phi(\vec{x}')$$

is valid to use.

Popular ones:

Linear $k(\vec{x}, \vec{x}') = \vec{x}^\mathsf{T} \vec{x}'$

Polynomial $k(\vec{x}, \vec{x}') = (c + \vec{x}^\mathsf{T} \vec{x}')^d$

[hyper-parameters: c, d]

Corresponds to all polynomial features up through degree d

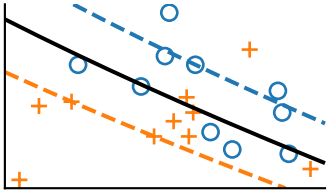
RBF $k(\vec{x}, \vec{x}') = e^{-\gamma \|\vec{x} - \vec{x}'\|^2}$

[hyper-parameter: γ]

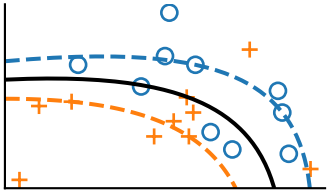
Corresponds to an infinite-dimensional feature space

Polynomial Kernel

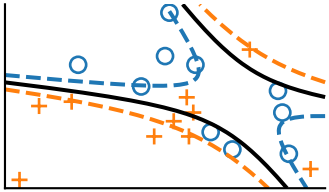
$$k(\vec{x}, \vec{x}') = (1 + \vec{x}^{\mathsf{T}} \vec{x}')^2$$



$C = 10$



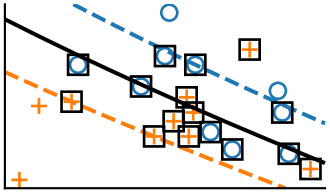
$C = 10^3$



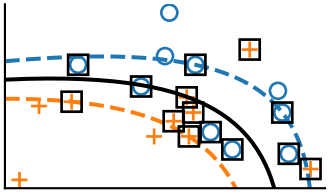
$C = 10^5$

Polynomial Kernel

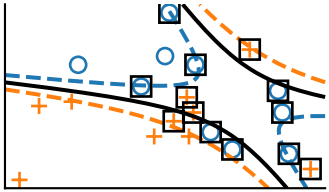
$$k(\vec{x}, \vec{x}') = (1 + \vec{x}^{\mathsf{T}} \vec{x}')^2$$



$C = 10$



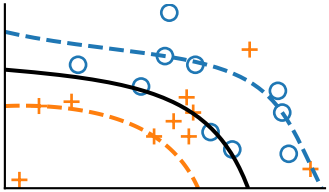
$C = 10^3$



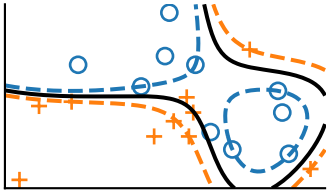
$C = 10^5$

RBF Kernel

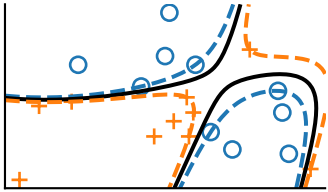
$$k(\vec{x}, \vec{x}') = e^{-\|\vec{x}-\vec{x}'\|^2}$$



$C = 10$



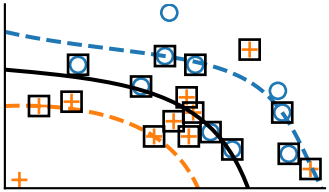
$C = 10^3$



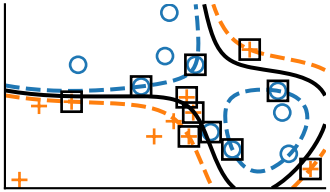
$C = 10^5$

RBF Kernel

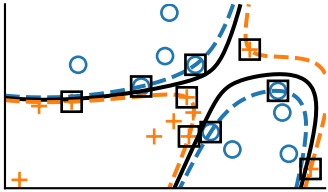
$$k(\vec{x}, \vec{x}') = e^{-\|\vec{x}-\vec{x}'\|^2}$$



$C = 10$



$C = 10^3$



$C = 10^5$

Notes on SVMs

- SVMs identify the points on the boundary: the support vectors
  - a way of figuring out "what to remember"
  - might be interesting examples to consider

Notes on SVMs

- SVMs identify the points on the boundary: the support vectors
  - a way of figuring out "what to remember"
  - might be interesting examples to consider
- Specific QP solvers for the SVM problem have been developed
  - often faster than a general QP solver
  - some can handle data that do not all fit in memory

Notes on Kernels

- $k(\vec{x}, \vec{x}')$ measures the similarity of $f(\vec{x})$ and $f(\vec{x}')$.
- Kernel can be calculated more efficiently than generating the features

Notes on Kernels

- $k(\vec{x}, \vec{x}')$ measures the similarity of $f(\vec{x})$ and $f(\vec{x}')$.
- Kernel can be calculated more efficiently than generating the features
- Many more kernels possible

Notes on Kernels

- $k(\vec{x}, \vec{x}')$ measures the similarity of $f(\vec{x})$ and $f(\vec{x}')$.
- Kernel can be calculated more efficiently than generating the features
- Many more kernels possible
- Kernels possible for non-vector data:
  - string kernel example: count of number of common substrings
  - graph kernel example: count of number of common subgraphs

Notes on Kernels

- $k(\vec{x}, \vec{x}')$ measures the similarity of $f(\vec{x})$ and $f(\vec{x}')$.
- Kernel can be calculated more efficiently than generating the features
- Many more kernels possible
- Kernels possible for non-vector data:
  - string kernel example: count of number of common substrings
  - graph kernel example: count of number of common subgraphs
- Kernels have their own hyper-parameters (in addition to C)

Notes on Kernels

- $k(\vec{x}, \vec{x}')$ measures the similarity of $f(\vec{x})$ and $f(\vec{x}')$.
- Kernel can be calculated more efficiently than generating the features
- Many more kernels possible
- Kernels possible for non-vector data:
  - string kernel example: count of number of common substrings
  - graph kernel example: count of number of common subgraphs
- Kernels have their own hyper-parameters (in addition to C)
- Kernels can be applied to almost every ML method to develop a non-linear version